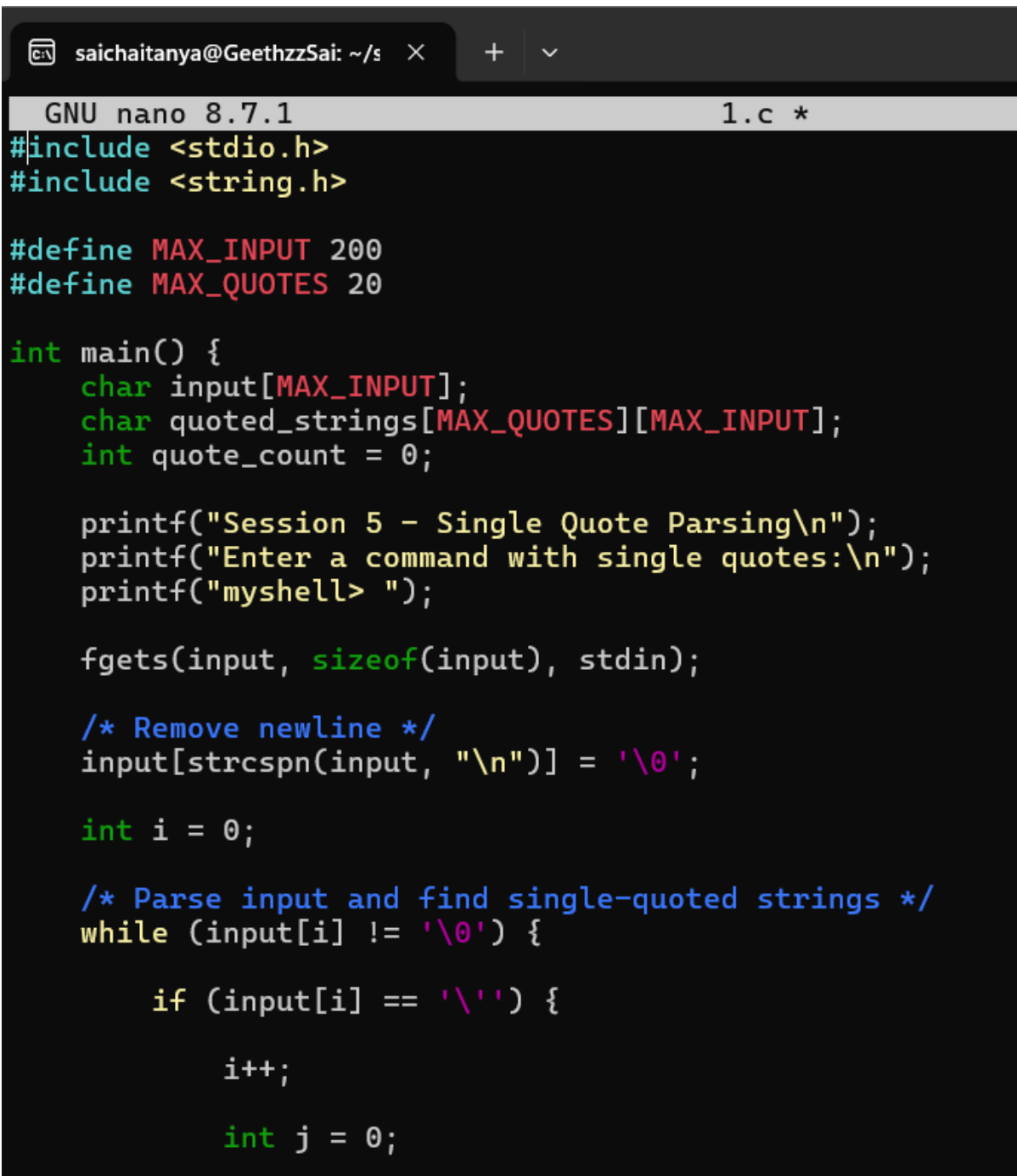


SESSION-5

Task 1: To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

CODE:A screenshot of a terminal window with a dark background. The window title bar shows 'saichaitanya@GeethzzSai: ~/s' and standard window controls. The terminal displays the GNU nano 8.7.1 editor interface. The code is a C program for parsing single-quoted strings. It includes <stdio.h> and <string.h>, defines MAX_INPUT as 200 and MAX_QUOTES as 20, and implements a main function that reads a line of input, removes the newline, and begins parsing for single-quoted strings. The code is color-coded: keywords in green, comments in blue, and strings/variables in red or yellow.

```
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 200
#define MAX_QUOTES 20

int main() {
    char input[MAX_INPUT];
    char quoted_strings[MAX_QUOTES][MAX_INPUT];
    int quote_count = 0;

    printf("Session 5 - Single Quote Parsing\n");
    printf("Enter a command with single quotes:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int i = 0;

    /* Parse input and find single-quoted strings */
    while (input[i] != '\0') {

        if (input[i] == '\') {

            i++;

            int j = 0;
```

```
/* Store everything until the closing quote */
while (input[i] != '\0' && input[i] != '\') {

    if (j < MAX_INPUT - 1) {
        quoted_strings[quote_count][j++] = input[i];
    }

    i++;
}

quoted_strings[quote_count][j] = '\0';
```

```
/* Check for closing quote */
if (input[i] == '\') {
    if (quote_count < MAX_QUOTES - 1) {
        quote_count++;
    }
    i++;
}
else {
    printf("Error: Unclosed single quote.\n");
    return 1;
}

} else {
    i++;
}

}
```

```
printf("\nQuoted strings found:\n");

if (quote_count == 0) {
    printf("No single-quoted strings found.\n");
}
else {
    for (int k = 0; k < quote_count; k++) {
        printf("Quoted String %d: [%s]\n",
            k + 1, quoted_strings[k]);
    }
}

printf("\nParsing validation:\n");

if (quote_count > 0) {
    printf("Single-quote parsing successful.\n");
}
else {
    printf("No quoted content to validate.\n");
}

return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai: ~/s × + v
saichaitanya@GeethzzSai:~$ cd skill_week3
saichaitanya@GeethzzSai:~/skill_week3$ nano 1.c
saichaitanya@GeethzzSai:~/skill_week3$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/skill_week3$ ./1
Session 5 - Single Quote Parsing
Enter a command with single quotes:
myshell> hello @world

Quoted strings found:
No single-quoted strings found.

Parsing validation:
No quoted content to validate.
saichaitanya@GeethzzSai:~/skill_week3$ |
```

Task 2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

CODE:

```
saichaitanya@GeethzzSai: ~/s × + v
GNU nano 8.7.1 2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 300
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

/* Expand environment variables such as $USER */
void expand_variable(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_INPUT - 1) {

        if (input[i] == '$') {
            i++;

            char variable[50];
            int k = 0;

            /* Read variable name */
            while (isalnum((unsigned char)input[i]) ||
                    input[i] == '_') {

                if (k < 49) {
                    variable[k++] = input[i];
                }

                i++;
            }
        }
    }
}
```

```
variable[k] = '\\0';

/* Get environment variable value */
char *value = getenv(variable);

if (value != NULL) {
    int v = 0;

    while (value[v] != '\\0' &&
           j < MAX_INPUT - 1) {
```

```
        output[j++] = value[v++];
    }
}

} else {
    output[j++] = input[i++];
}

output[j] = '\\0';
}

int main() {

    char input[MAX_INPUT];
    char expanded[MAX_INPUT];

    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];

    int token_count = 0;
    int i = 0;
```

```
printf("Session 5 - Double Quote Parsing\n");
printf("Double quotes preserve spaces and allow variable expansion.\n");
printf("Enter a command:\n");
printf("myshell> ");

fgets(input, sizeof(input), stdin);

/* Remove newline */
input[strcspn(input, "\n")] = '\0';

/* Handle empty command */
if (strlen(input) == 0) {
    printf("Empty command.\n");
    return 0;
}

/* Parse input */
while (input[i] != '\0' &&
       token_count < MAX_TOKENS) {

    /* Skip spaces */
    while (isspace((unsigned char)input[i])) {
        i++;
    }

    if (input[i] == '\0') {
        break;
    }

    int j = 0;
```

```

/* Double-quoted string */
if (input[i] == '"') {

    i++;

    while (input[i] != '\0' &&
           input[i] != '"') {

        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[token_count][j++] = input[i];
        }

        i++;
    }

    /* Check closing quote */
    if (input[i] == '"') {
        i++;
    } else {
        printf("Error: Unclosed double quote.\n");
        return 1;
    }

    tokens[token_count][j] = '\0';
    token_count++;

}

/* Normal token */
else {

```

```

    while (input[i] != '\0' &&
           !isspace((unsigned char)input[i]) &&
           input[i] != '"') {

        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[token_count][j++] = input[i];
        }

        i++;
    }

    tokens[token_count][j] = '\0';

```

```

        if (j > 0) {
            token_count++;
        }
    }
}

/* Display tokens before expansion */
printf("\nTokens before variable expansion:\n");

for (int k = 0; k < token_count; k++) {
    printf("Token %d: [%s]\n", k + 1, tokens[k]);
}

```

```

/* Variable expansion */
printf("\nVariable expansion:\n");

for (int k = 0; k < token_count; k++) {
    expand_variable(tokens[k], expanded);

    printf("Token %d: [%s]\n",
        k + 1, expanded);
}

/* Validation */
printf("\nParsing validation:\n");
printf("Double-quote parsing successful.\n");
printf("Spaces inside quotes were preserved.\n");
printf("Variable expansion was processed.\n");

return 0;
}

```

OUTPUT:

```
No quoted content to validate.  
saichaitanya@GeethzzSai:~/skill_week3$ nano 2.c  
saichaitanya@GeethzzSai:~/skill_week3$ gcc 2.c -o 2  
saichaitanya@GeethzzSai:~/skill_week3$ ./2  
Session 5 - Double Quote Parsing  
Double quotes preserve spaces and allow variable expansion.  
Enter a command:  
myshell> hello world  
  
Tokens before variable expansion:  
Token 1: [hello]  
Token 2: [world]  
  
Variable expansion:  
Token 1: [hello]  
Token 2: [world]  
  
Parsing validation:  
Double-quote parsing successful.  
Spaces inside quotes were preserved.  
Variable expansion was processed.  
saichaitanya@GeethzzSai:~/skill_week3$ |
```

SESSION-6

Task 1: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 300

int main() {
    char input[MAX_INPUT];
    char output[MAX_INPUT];

    int i = 0;
    int j = 0;
    int escaped = 0;

    printf("Session 6 - Escape Sequence Parsing\n");
    printf("Enter a command with escaped spaces or special characters:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    /* Handle empty input */
    if (strlen(input) == 0) {
        printf("Empty input.\n");
        return 0;
    }

    /* Process escape sequences */
    while (input[i] != '\0' && j < MAX_INPUT - 1) {

        if (escaped) {
            output[j++] = input[i];
            escaped = 0;
        }
        else if (input[i] == '\\') {
            escaped = 1;
        }
        else {
            output[j++] = input[i];
        }

        i++;
    }
}
```

```

/* Check if input ends with escape character */
if (escaped) {
    printf("Warning: Escape character at the end of input.\n");
}

output[j] = '\0';

printf("\nOriginal input:\n");
printf("%s\n", input);

printf("\nProcessed input:\n");
printf("%s\n", output);

printf("\nParser validation:\n");
printf("Escape sequences processed successfully.\n");
printf("Escaped characters were preserved.\n");

return 0;
}

```

OUTPUT:

```

saichaitanya@GeethzzSai:~/skill_week3$ nano 3.c
saichaitanya@GeethzzSai:~/skill_week3$ gcc 3.c -o 3
saichaitanya@GeethzzSai:~/skill_week3$ ./3
Session 6 - Escape Sequence Parsing
Enter a command with escaped spaces or special characters:
myshell> ls

Original input:
ls

Processed input:
ls

Parser validation:
Escape sequences processed successfully.
Escaped characters were preserved.
saichaitanya@GeethzzSai:~/skill_week3$ |

```

Task 2: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Session 6 - Process Execution\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing: ls -l\n");

        execlp("ls", "ls", "-l", NULL);

        /* This runs only if exec fails */
        perror("exec failed");
        exit(EXIT_FAILURE);
    }

    else {
        /* Parent process */
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("\nChild exited with status: %d\n",
                WEXITSTATUS(status));
        }
        else {
            printf("\nChild terminated abnormally.\n");
        }
    }
}
```

```

        printf("Parent process completed.\n");
    }

    return 0;
}

```

OUTPUT:

```

saichaitanya@GeethzzSai:~/skill_week3$ nano 4.c
saichaitanya@GeethzzSai:~/skill_week3$ gcc 4.c -o 4
saichaitanya@GeethzzSai:~/skill_week3$ ./4
Session 6 - Process Execution

Parent Process
Parent PID: 1034
Child PID: 1035
Parent waiting for child...

Child Process
Child PID: 1035
Parent PID: 1034
Executing: ls -l
total 80
-rwxr-xr-x 1 saichaitanya saichaitanya 16168 Aug 15 07:13 1
-rw-r--r-- 1 saichaitanya saichaitanya  1823 Aug 15 07:13 1.c
-rwxr-xr-x 1 saichaitanya saichaitanya 16304 Aug 15 07:16 2
-rw-r--r-- 1 saichaitanya saichaitanya  3777 Aug 15 07:16 2.c
-rwxr-xr-x 1 saichaitanya saichaitanya 16168 Aug 15 07:18 3
-rw-r--r-- 1 saichaitanya saichaitanya  1337 Aug 15 07:18 3.c
-rwxr-xr-x 1 saichaitanya saichaitanya 16344 Aug 15 07:21 4
-rw-r--r-- 1 saichaitanya saichaitanya  1198 Aug 15 07:20 4.c

Child exited with status: 0
Parent process completed.
saichaitanya@GeethzzSai:~/skill_week3$ |

```